Data Analysis

Frequent Patterns

Section 7

Eng. Asmaa Fathy

Frequent Pattern

- Frequent-pattern mining is a Data Mining task that aims to discover patterns or associations in a dataset that occur frequently. These patterns can be used to uncover relationships among items in the dataset and can be used for tasks such as market basket analysis, recommendation systems, and fraud detection.
- FREQUENT ITEM SET AND ASSOCIATION RULES.
- Apriori Algorithm.
- FP-Growth Algorithm.

Frequent Itemset and Association Rules

- Itemset = A group of products/items.
- Frequent = Appears frequently
- Frequent Itemset : Things that are often bought together
- Example: In a supermarket If we find that: Many people buy (bread + cheese) Or (chips + soda) → This is called Frequent Itemsets.
- Frequent itemset mining identifies sets of items that frequently co-occur in transactions.

- Association Rules are rules or relationships between things.
- They look like this: If A happens → B will likely happen
- Example: If someone buys milk → they will likely buy cornflakes, If someone buys a mobile phone → they might buy a case → This is called an Association Rule

Transactional Data means data containing purchase transactions.
For example:

Transaction Number	Products
1	Bread, Cheese
2	Bread, Milk
3	Cheese, Milk
4	Bread, Cheese

Most Important Key Terms

1. Support: is a metric used to measure how frequently a group of elements occur together in a database.

   Support (A → B) = $\frac{\textit{The number of transactions that contain A and B}}{\textit{The Total Number of Transactions}}$

   Example: If we have 5 transactions, and 3 of them contain both milk and bread.
   Support = 3/5 = 0.6 (means **60%** of transactions contain both milk and bread).

2. Confidence (in the Rules): A measure that determines how likely element B is to occur when element A is present.

   Confidence (A → B) = = $\frac{\textit{The number of transactions that contain A and B}}{\textit{The number of transactions that contain A}}$

   Example: If 4 transactions contain milk, and 3 of them contain bread and milk.
   Confidence = ¾ = 0.75 (means **75%** of transactions that contain milk also contain bread).

Most Important Key Terms

3. Lift (The True Strength of the Relationship): "Is this relationship real or just a coincidence?“

Example: If one person is famous (Bread) and another is famous (Butter), they might look a lot together, but each is famous on their own. Lift is what reveals the truth.

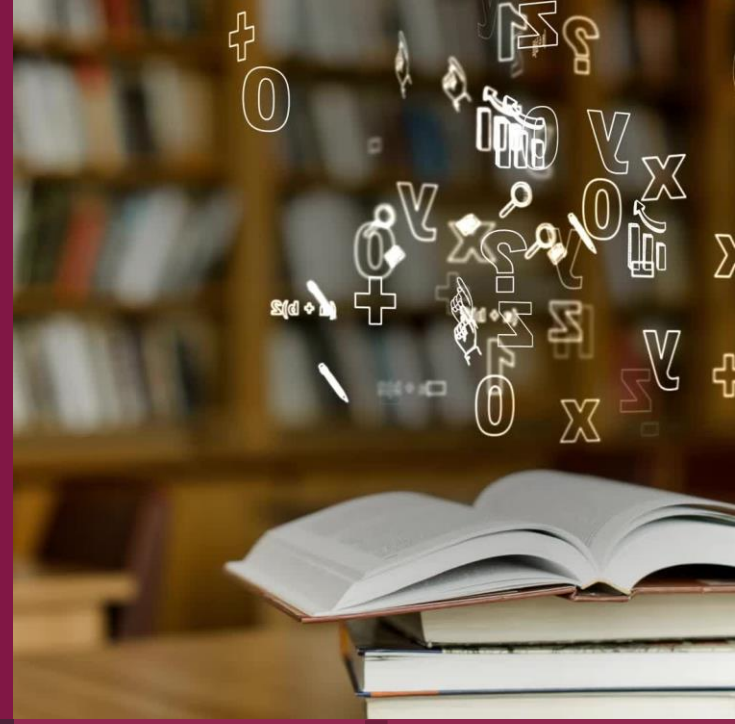
- If lift > 1 → Strong relationship (They happen together more often than expected)
- If lift = 1 → No relationship (coincidence)
- If lift < 1 → Weak or negative relationship

mlxtend package

- This is a Python library that simplifies this process
- We use:

1. Apriori: To retrieve the Frequent Itemsets
2. association_rules: To retrieve the rules

```
!pip install mlxtend
```



```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder

# Sample dataset (list of transactions)
dataset = [
['Milk', 'Bread', 'Eggs'],
['Bread', 'Eggs', 'Butter'],
['Milk', 'Eggs'],
['Milk', 'Bread', 'Eggs', 'Butter'],
['Milk', 'Bread'],
['Bread', 'Eggs'],
['Milk', 'Bread', 'Butter'],
]

# Convert the dataset into a one-hot encoded format
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)
print(df)
```

	Bread	Butter	Eggs	Milk
0	True	False	True	True
1	True	True	True	False
2	False	False	True	True
3	True	True	True	True
4	True	False	False	True
5	True	False	True	False
6	True	True	False	True

How to recognize frequent itemsets based on minimum support using the Apriori algorithm in Python?

```python
from mlxtend.frequent_patterns import apriori , association_rules

# Define the minimum support threshold
min_support = 0.4

# Perform frequent itemset mining using Apriori
frequent_itemsets = apriori(df, min_support=min_support, use_colnames=True)
print(frequent_itemsets)
```

	support	itemsets
0	0.857143	(Bread)
1	0.428571	(Butter)
2	0.714286	(Eggs)
3	0.714286	(Milk)
4	0.428571	(Butter, Bread)
5	0.571429	(Bread, Eggs)
6	0.571429	(Milk, Bread)
7	0.428571	(Milk, Eggs)

Detecting Association Rules

```python
# Generate association rules with minimum confidence threshold (e.g., 0.6)
min_confidence = 0.6

association_rules_df = association_rules(frequent_itemsets, metric="confidence", min_threshold=min_confidence)

print("\nAssociation Rules:")
print(association_rules_df[['antecedents', 'consequents','support', 'confidence','lift']])
```

```
Association Rules:
  antecedents consequents   support  confidence      lift
0    (Butter)     (Bread)  0.428571    1.000000  1.166667
1     (Bread)      (Eggs)  0.571429    0.666667  0.933333
2      (Eggs)     (Bread)  0.571429    0.800000  0.933333
3      (Milk)     (Bread)  0.571429    0.800000  0.933333
4     (Bread)      (Milk)  0.571429    0.666667  0.933333
5      (Milk)      (Eggs)  0.428571    0.600000  0.840000
6      (Eggs)      (Milk)  0.428571    0.600000  0.840000
```

Support: "How widespread is this need?"

Confidence: "If A happens, how often does B happen?"

Lift: "Is this relationship real or a coincidence?"

Apriori Algorithm VS FP-Growth Algorithm

Apriori	FP-Growth
We try all the possibilities one by one, but in a slightly smarter way.	We won't try everything. We'll organize the data first.
1. Identify the recurring elements. 2. Create larger groups from them. 3. Remove the weak elements. 4. Develop rules. 5. Use confidence + lift.	1. It builds a tree called FP-Tree. 2. It organizes all the recurring data into it. 3. Then it retrieves the frequent itemsets directly
Important drawback: If the data is very large → It becomes slow because it tries many things	Advantages: Much faster than Apriori. Doesn't repeat calculations

Example (Transactional Data)

- We have 5 shopping baskets:

Transaction	Items
T1	Bread, Milk
T2	Bread, Diapers, Beer
T3	Milk, Diapers, Beer, Cola
T4	Bread, Milk, Diapers, Beer
T5	Bread, Milk, Diapers

min_support = 0.6
min_confidence = 0.7

Step 1: Calculate Support for each item

1-itemsets

- Number of transactions = 5
- Bread → found in (T1, T2, T4, T5) = 4/5 = 0.8 ✔
- Milk → 4/5 = 0.8 ✔
- Diapers → 4/5 = 0.8 ✔
- Beer → 3/5 = 0.6 ✔
- Cola → 1/5 = 0.2 ✘

2-itemsets

- (Bread, Milk) found in T1, T4, T5 = 3/5 = 0.6 ✔
- (Bread, Diapers) T2, T4, T5 = 3/5 = 0.6 ✔
- (Milk, Diapers) T3, T4, T5 = 3/5 = 0.6 ✔
- (Diapers, Beer) T2, T3, T4 = 3/5 = 0.6 ✔
- (Milk, Beer) T3, T4 = 2/5 = 0.4 ✘ (Remove)

3-itemsets

- (Bread, Milk, Diapers) T4, T5 = 2/5 = 0.4 ✘
- (Milk, Diapers, Beer) T3, T4 = 2/5 = 0.4 ✘
- That's it, we'll stop here

Step 2: Find the Association Rules

Rule 1: Bread → Milk

1. **Calculate Confidence:**

Bread in 4 transactions
Bread + Milk in 3 transactions
[Confidence = 3/4 = 0.75] Good

2. **Calculate Lift:**

P(Milk) = 4/5 = 0.8
P(Bread) = 0.8
Expected = 0.8 × 0.8 = 0.64
Actual = 3/5 = 0.6
[Lift = 0.6 / 0.64 = 0.94] ✗
"Even if they come together, it's probably because each is common on its own, not a strong relationship."

Rule 2: Diapers → Beer

1. **Calculate Confidence:**

Diapers in 4 transactions
With Beer in 3 transactions
[Confidence = 3/4 = 0.75] Good

2. **Calculate Lift:**

P(Diapers) = 0.8
P(Beer) = 0.6
Expected = 0.8 x 0.6 = 0.48
Actual = 0.6
[Lift = 0.6 / 0.48 = 1.25] ✓

Apriori Algorithm

```python
import pandas as pd
import numpy as np
from mlxtend.preprocessing import TransactionEncoder

# Set a random seed for reproducibility
np.random.seed(42)

# Number of records (transactions) in the dataset
num_records = 1000

# Number of unique items in the grocery store
num_items = 10

# Generate the dummy dataset
transactions = []

for _ in range(num_records):
    num_items_in_transaction = np.random.randint(1, num_items + 1)
    items = np.random.choice(range(1, num_items + 1), num_items_in_transaction, replace=False)
    transactions.append([f"Item{item}" for item in items])

# Convert the dataset into a one-hot encoded format
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(te_ary, columns=te.columns_)
print(df_encoded)
```

```
     Item1  Item10  Item2  Item3  Item4  Item5  Item6  Item7  Item8  Item9
0     True    True   True   True  False  False   True   True  False   True
1     True    True   True   True  False  False   True   True   True   True
2    False   False  False   True  False  False  False   True  False  False
3     True    True   True   True   True   True   True   True   True   True
4     True    True   True   True   True   True   True  False   True   True
..     ...     ...    ...    ...    ...    ...    ...    ...    ...    ...
995   True   False   True   True  False  False  False  False  False   True
996  False   False  False  False  False  False  False   True  False  False
997   True   False  False   True   True   True  False   True  False   True
998  False    True  False   True  False  False  False   True  False  False
999  False   False  False  False   True  False  False   True   True   True

[1000 rows x 10 columns]
```

Apriori Algorithm

```python
from mlxtend.frequent_patterns import apriori

# Define the minimum support threshold
min_support = 0.35

# Perform frequent itemset mining using Apriori
frequent_itemsets = apriori(df_encoded, min_support=min_support,use_colnames=True)
print("FrequentItemsets:")
print(frequent_itemsets)
```

```
FrequentItemsets:
    support         itemsets
0     0.530          (Item1)
1     0.546         (Item10)
2     0.543          (Item2)
3     0.545          (Item3)
4     0.527          (Item4)
5     0.540          (Item5)
6     0.533          (Item6)
7     0.532          (Item7)
8     0.533          (Item8)
9     0.545          (Item9)
10    0.358   (Item1, Item5)
11    0.358  (Item10, Item2)
12    0.359  (Item10, Item3)
13    0.351  (Item10, Item4)
14    0.353  (Item10, Item5)
15    0.364  (Item10, Item9)
16    0.351   (Item4, Item2)
17    0.357   (Item6, Item2)
18    0.352   (Item8, Item2)
19    0.362   (Item9, Item2)
20    0.354   (Item3, Item4)
21    0.362   (Item3, Item5)
22    0.357   (Item6, Item3)
23    0.360   (Item3, Item7)
24    0.362   (Item8, Item3)
25    0.356   (Item3, Item9)
26    0.358   (Item5, Item4)
27    0.350   (Item8, Item4)
28    0.352   (Item9, Item4)
29    0.352   (Item6, Item5)
30    0.352   (Item8, Item5)
31    0.356   (Item5, Item9)
32    0.354   (Item8, Item6)
33    0.355   (Item8, Item9)
```

```python
from mlxtend.frequent_patterns import association_rules

#Generate association rules with minimum confidence threshold (e.g.,0.5)
min_confidence=0.67

association_rules_df=association_rules(frequent_itemsets,metric="confidence",min_threshold=min_confidence)
print("\nAssociationRules:")
print(association_rules_df[['antecedents', 'consequents','support', 'confidence','lift']])
```

```
AssociationRules:
  antecedents consequents  support  confidence      lift
0     (Item1)     (Item5)    0.358    0.675472  1.250874
1     (Item4)     (Item3)    0.354    0.671727  1.232526
2     (Item5)     (Item3)    0.362    0.670370  1.230037
3     (Item7)     (Item3)    0.360    0.676692  1.241636
4     (Item8)     (Item3)    0.362    0.679174  1.246192
5     (Item4)     (Item5)    0.358    0.679317  1.257994
```

FP-Growth Algorithm

```python
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth,association_rules

# Sample dataset (list of transactions)
dataset=[
['Milk','Bread','Eggs'],
['Bread','Eggs','Butter'],
['Milk','Eggs'],
['Milk','Bread','Eggs','Butter'],
['Milk','Bread'],
['Bread','Eggs'],
['Milk','Bread','Butter'],
]

# Convert the dataset into a one-hot encoded format
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)
print(df)
```

```
   Bread  Butter   Eggs   Milk
0   True   False   True   True
1   True    True   True  False
2  False   False   True   True
3   True    True   True   True
4   True   False  False   True
5   True   False   True  False
6   True    True  False   True
```

```python
# Apply FP-growth to find frequent itemsets
frequent_itemsets = fpgrowth(df, min_support=0.4, use_colnames=True)
print(frequent_itemsets)
```

```
    support         itemsets
0  0.857143          (Bread)
1  0.714286           (Milk)
2  0.714286           (Eggs)
3  0.428571         (Butter)
4  0.571429    (Milk, Bread)
5  0.571429    (Bread, Eggs)
6  0.428571     (Milk, Eggs)
7  0.428571  (Butter, Bread)
```

```python
# Generate association rules with minimum confidence threshold (e.g., 0.6)
min_confidence = 0.6

association_rules_df = association_rules(frequent_itemsets, metric="confidence", min_threshold=min_confidence)
print("\nAssociation Rules:")
print(association_rules_df[['antecedents', 'consequents','support', 'confidence','lift']])
```

```
Association Rules:
  antecedents consequents   support  confidence      lift
0      (Milk)     (Bread)  0.571429    0.800000  0.933333
1     (Bread)      (Milk)  0.571429    0.666667  0.933333
2     (Bread)      (Eggs)  0.571429    0.666667  0.933333
3      (Eggs)     (Bread)  0.571429    0.800000  0.933333
4      (Milk)      (Eggs)  0.428571    0.600000  0.840000
5      (Eggs)      (Milk)  0.428571    0.600000  0.840000
6    (Butter)     (Bread)  0.428571    1.000000  1.166667
```

Task

- Create a dataset of **1000 transactions** representing a grocery store. Each transaction contains random items from **15 unique items (Item1 → Item15)**.
- Apply Apriori Algorithm and Generate Association Rules (Use minimum support = **0.3 and** confidence threshold = **0.6**).
- Apply FP-Growth Algorithm and Generate Association Rules (Use minimum support = **0.3 and** confidence threshold = **0.6**)

Thank You